

Programação Orientada a Objetos

Capítulo 2 — Paradigma de Programação Orientada a Objetos

Aula 02

O que veremos hoje?

- 1 Algoritmos: conceito e importância
- 2 Programação e gerações de linguagens
- 3 Programação Estruturada
- 4 Programação Orientada a Objetos (POO)
- 5 Estruturada vs. Orientada a Objetos — comparativo
- 6 **Dinâmicas & Atividades práticas**



2.1 Algoritmos

Sequências finitas de etapas para executar tarefas

O que são algoritmos?

- Algoritmo: sequência finita de etapas para executar uma tarefa pré-determinada;
- Executamos algoritmos no cotidiano: tomar banho, escovar dentes, atravessar a rua, conferir troco;
- Algoritmos computacionais também nos cercam: *GPS* calcula menor rota, e-commerce detecta fraudes;
- Diferença crucial: instruções para humanos podem ser livres; para computadores exigem precisão perfeita;
- O computador executa exatamente o que lhe for indicado, linha a linha, sem questionamentos.

Precisão computacional *vs.* humana

- Um ser humano tem capacidade cognitiva de interpretar e adaptar instruções ambíguas;
- O computador não tem bom senso: se receber uma instrução sem fim, a executa até ser forçado a parar;
- Loops infinitos são um erro comum causado por falta de precisão na descrição do algoritmo;
- Conclusão: a qualidade do algoritmo depende diretamente da clareza e detalhamento do programador;
- Algoritmos bem escritos são a base de qualquer software, independentemente do paradigma.

Dinâmica 1 — Algoritmo do Dia a Dia

- Em duplas, escolham uma atividade cotidiana simples (ex.: fazer café, sacar dinheiro no caixa eletrônico);
- Escrevam o algoritmo com pelo menos 8 passos sequenciais;
- Agora, troquem os algoritmos entre as duplas;
- A dupla que recebeu deve executar **LITERALMENTE** cada passo, como um computador faria;
- Identifiquem: onde a instrução foi ambígua? O que um computador faria de errado?
- Tempo: 7 minutos. Discussão: 3 minutos.

Reflexões — Dinâmica 1

- Exemplo 'Fazer café': se não disse 'colocar filtro no suporte', o computador tentaria jogar pó direto na jarra;
- Passos como 'colocar um pouco de açúcar' são ambíguos — quanto é 'um pouco'? O computador precisa de '2 colheres';
- Omitir 'ligar a cafeteira' faz o computador esperar infinitamente pela água esquentar;
- Lição: algoritmos computacionais exigem cada passo descrito com precisão absoluta.



2.2 Programação

Das linguagens de máquina ao alto nível

O que é programação?

- Programação: processo de transcrição dos algoritmos para o computador;
- Um sistema de computador é composto por algoritmos que se complementam para executar processos;
- Para desenvolver sistemas, é necessário escolher uma linguagem de programação e uma metodologia;
- As linguagens evoluíram ao longo das décadas, dividindo-se em três gerações principais.

As três gerações de linguagens

- 1ª Geração — Linguagem de máquina: sequências numéricas (binário), complexa e custosa, linguagem natural do hardware;
- 2ª Geração — Linguagem assembler: abreviações em inglês substituem os números, mais produtiva, mas requer tradutores;
- 3ª Geração — Linguagem de alto nível: semelhante à comunicação humana, baseada no inglês, usa expressões matemáticas;
- Evolução permitiu ao programador focar em funcionalidades, não na complexidade da linguagem;
- Exemplos atuais de alto nível: *Java*, *Python*, *C#*, *JavaScript* — todas operam com paradigmas como OO.

Dinâmica 2 — Traduzindo entre Gerações

- Em grupos de 3, imaginem a instrução: 'Some os números 5 e 3 e mostre o resultado'.
- Como seria essa instrução em cada geração?
- Geração 1 (máquina): tentem representar com números fictícios (ex.: 0010 0101 0011 ...).
- Geração 2 (assembler): usem mnemônicos (ex.: `LOAD 5, ADD 3, PRINT`).
- Geração 3 (alto nível): escrevam em pseudocódigo ou qualquer linguagem que conheçam.
- Tempo: 5 minutos. Qual geração foi mais intuitiva?

Exemplos de resposta — Dinâmica 2

- Máquina: 00101000 00000101 00110000 00000011 01010000 — incompreensível sem documentação.
- *Assembler*: LOAD R1, 5 / ADD R1, 3 / MOV R2, R1 / PRINT R2 — legível mas verboso.
- Alto nível: resultado = 5 + 3; print(resultado) — quase uma frase em inglês.
- A cada geração, o programador ganha produtividade e se aproxima do raciocínio humano.



2.3 Prog. Estruturada

O paradigma clássico: execução sequencial de instruções

O que é um paradigma de programação?

- Paradigma = forma como pensamos ao programar. Define desde o levantamento de requisitos até a execução;
- Existem vários: programação estruturada, orientada a objetos, concorrente, funcional, etc;
- Cada paradigma exige do programador conhecimentos e formas de raciocínio específicos;
- Linguagens orientadas a paradigma obrigam o uso de um estilo; outras permitem flexibilidade;
- O paradigma influencia toda a cadeia: análise → projeto → codificação → uso pelo usuário.

Características da Programação Estruturada

- Método mais clássico e geralmente o primeiro paradigma aprendido por programadores;
- Baseia-se na execução de instruções em ordem sequencial — passo a passo;
- Permite agrupamento de código em funções ou procedimentos reutilizáveis;
- Exemplo: calcular média → receber 2 números → somar → dividir por 2 (um passo de cada vez);
- Mesmo com funções, o código é executado linha por linha de forma sequencial;
- Aprendizado mais fácil inicialmente por ser mais 'ordenada' e linear.

Dinâmica 3 — Pensando de Forma Estruturada

- Individualmente, escreva um algoritmo estruturado para: 'Calcular o salário líquido de um funcionário';
- Dados: salário bruto, desconto INSS (11%), desconto IR (15% sobre o que resta após INSS);
- Escreva passo a passo, de forma sequencial, como na programação estruturada;
- Não use classes nem objetos — apenas variáveis e passos em sequência;
- Tempo: 5 minutos.

Resposta possível — Dinâmica 3

- Passo 1: Receber `salarioBruto`;
- Passo 2: `descontoINSS = salarioBruto × 0.11`;
- Passo 3: `baseIR = salarioBruto - descontoINSS`;
- Passo 4: `descontoIR = baseIR × 0.15`;
- Passo 5: `salarioLiquido = baseIR - descontoIR`;
- Passo 6: Exibir `salarioLiquido`;
- Note: tudo é sequencial, sem troca de mensagens entre objetos — uma instrução após a outra.



2.4 Programação OO

Classes, objetos e troca de mensagens desde os anos 1960

Origens e popularização da POO

- Surgiu na década de 1960 com a linguagem Simula 67 — uma das primeiras com conceito de classes;
- Tornou-se popular na década de 1980 e é um dos paradigmas mais utilizados hoje;
- Grande facilitador: reaproveitamento de código por meio de herança e composição;
- Desde a análise de requisitos, o software é pensado em termos de classes e objetos do mundo real;
- Reutilização de objetos ocorre desde o início do desenvolvimento, acelerando entregas.

Como funciona a execução na POO?

- A execução não é estritamente sequencial como na programação estruturada;
- O motor do sistema é a troca de mensagens entre objetos — um objeto chama métodos de outro;
- A sequência de instruções muda conforme as requisições feitas entre os objetos em execução;
- Cada objeto pode ter seus próprios métodos sendo invocados em qualquer ordem necessária;
- Resultado: um fluxo de execução dinâmico, mais próximo da forma como processos reais acontecem.

Dinâmica 4 — Mesmo Problema, Pensamento OO

- Retome o cálculo do salário líquido da Dinâmica 3;
- Agora, modele a solução usando orientação a objetos:
 - Quais classes você criaria? (ex.: Funcionario, FolhaPagamento, Imposto...)
 - Quais atributos e métodos cada classe teria?
 - Como os objetos trocariam mensagens para calcular o salário líquido?
- Tempo: 7 minutos. Discutam em duplas.

Exemplo de resposta — Dinâmica 4

- Classe Funcionario: atributos (nome, salarioBruto). Método: `getSalarioBruto()`;
- Classe Imposto: atributos (aliquotaINSS, aliquotaIR). Métodos: `calcularINSS(bruto)`, `calcularIR(base)`;
- Classe FolhaPagamento: método `calcularLiquido(funcionario, imposto)` — chama métodos dos outros objetos;
- Fluxo: FolhaPagamento pede salário ao Funcionario → envia ao Imposto → recebe descontos → calcula líquido;
- Diferença: os objetos interagem por mensagens, cada um com sua responsabilidade específica.



2.5 Estruturada *vs.* OO

Semelhanças, diferenças e quando usar cada uma

Comparativo: Estruturada vs. Orientada a Objetos

Prog. Estruturada

- Execução sequencial de instruções;
- Agrupamento por funções/procedimentos;
- Código executado linha por linha;
- Mais fácil de aprender inicialmente;
- Pensamento linear e ordenado;

Prog. Orientada a Objetos

- Execução baseada em troca de mensagens;
- Agrupamento por classes e objetos;
- Fluxo dinâmico entre objetos;
- Exige adaptação na forma de pensar;
- Mais próxima da realidade empresarial;

O que têm em comum e o que diferencia?

- Ambos paradigmas usam instruções escritas linha a linha com rigor de sintaxe;
- Desempenho depende mais da linguagem e do ambiente do que do paradigma em si;
- OO tende a segmentar o código em mais partes e faz mais chamadas de classes;
- A qualidade do algoritmo do programador é o fator mais determinante para eficiência;
- A migração entre os paradigmas não costuma ser difícil — ambos usam instruções escritas;
- Linguagens multiparadigma (ex.: *Python*, *C++*) permitem escolher o estilo mais confortável.

Dinâmica 5 — Debate: Qual Paradigma Usar?

- Dividam a turma em dois times: Time Estruturada vs. Time OO;
- Cenário: uma startup precisa criar um app de *delivery* de comida;
- Cada time deve argumentar por que seu paradigma é melhor para esse projeto;
- Considerem: velocidade de desenvolvimento, manutenção futura, tamanho da equipe, reutilização;
- Tempo: 5 minutos de preparação + 3 minutos por time para apresentar argumentos;
- Professor media o debate e destaca pontos fortes de cada lado ao final.

"O paradigma de OO vê o mundo como uma coleção de objetos que interagem entre si, tornando a programação muito mais próxima à realidade do cotidiano."

Pontos-chave da aula

- Algoritmos são sequências finitas de etapas — base de todo software;
- Linguagens evoluíram: máquina → *assembler* → alto nível;
- Prog. Estruturada: execução sequencial, agrupamento por funções;
- POO: surgiu com *Simula 67*, baseia-se em classes, objetos e troca de mensagens;
- OO não é necessariamente mais rápida, mas é mais próxima da realidade empresarial.